

O **JUnit** é um dos frameworks mais usados para **testes unitários em Java**, sendo praticamente padrão no ecossistema da linguagem.

1) O que é o JUnit?

É uma biblioteca que permite:

- Criar testes automatizados
- Validar o comportamento de métodos/classes
- Detectar erros rapidamente
- Garantir que mudanças no código não quebrem funcionalidades existentes

2) Para que ele serve?

Imagine que você tem um método:

```
public int somar(int a, int b) {  
    return a + b;  
}
```

Com JUnit, você consegue testar automaticamente:

```
@Test  
void deveSomarCorretamente() {  
    assertEquals(4, somar(2, 2));  
}
```

➡ Sempre que rodar o teste, o JUnit verifica se o resultado está correto.

3) Principais características

✓ Simples de usar

- Poucas anotações (@Test, @BeforeEach, etc.)

✓ Automatização

- Executa centenas de testes em segundos

✓ Integração com IDEs

- Funciona direto no:
 - IntelliJ
 - Eclipse
 - VS Code

✓ Base para TDD

- Muito usado com **Test Driven Development**

4) Principais recursos

♦ Anotações

- `@Test` → define um teste
- `@BeforeEach` → executa antes de cada teste
- `@AfterEach` → executa depois

♦ Asserts (validações)

```
assertEquals(10, valor);  
assertTrue(condicao);  
assertNotNull(obj);
```

Eles verificam se o resultado esperado está correto.

♦ Testes de exceção

```
assertThrows(IllegalArgumentException.class, () -> {  
    metodoQueDeveFalhar();  
});
```

♦ Testes parametrizados

Permitem testar vários dados com menos código:

```
@ParameterizedTest  
@ValueSource(ints = {1, 2, 3})  
void teste(int numero) {  
    assertTrue(numero > 0);  
}
```

}

5) Versões

♦ JUnit 4 (mais antigo)

- Muito usado ainda em projetos legados

♦ JUnit 5 (atual)

- Mais moderno e flexível
- Separado em módulos (Jupiter, Platform)

6) Por que usar?

- ✓ Evita bugs em produção
- ✓ Facilita manutenção do código
- ✓ Documenta o comportamento do sistema
- ✓ Dá segurança para refatorar

7) Criar projeto com suporte a testes

1. File → New Project
2. Escolha **Maven**
3. Marque:
 - ✓ Create from archetype (opcional)
4. Configure:
 - groupId: `com.exemplo`
 - ArtifactId: `projeto-teste`

Abra o arquivo `pom.xml` e adicione dependência do Junit:

```
<dependencies>
```

```
  <dependency>
```

```
    <groupId>org.junit.jupiter</groupId>
```

```
    <artifactId>junit-jupiter</artifactId>
```

```
    <version>5.10.0</version>
```

```
        <scope>test</scope>
    </dependency>
</dependencies>
```

Ativar execução do JUnit 5

Ainda no `pom.xml`, adicione:

```
<build>
    <plugins>
        <plugin>
            <groupId>org.apache.maven.plugins</groupId>
            <artifactId>maven-surefire-plugin</artifactId>
            <version>3.0.0</version>
        </plugin>
    </plugins>
</build>
```

8) Estrutura correta do projeto

O IntelliJ já cria automaticamente:

```
src
├── main/java    → código da aplicação
└── test/java    → testes
```

9) Criar primeiro teste

Classe de exemplo:

```
public class Calculadora {  
  
    public int somar(int a, int b) {  
  
        return a + b;  
  
    }  
  
}
```

Criando o teste:

1. Clique com botão direito na classe
2. Generate → Test
3. Escolha:
 - ✓ JUnit5
4. Clique em OK

Teste gerado:

```
import org.junit.jupiter.api.Test;  
  
import static org.junit.jupiter.api.Assertions.*;  
  
class CalculadoraTest {  
  
    @Test  
  
    void deveSomar() {  
  
        Calculadora calc = new Calculadora();  
  
        int resultado = calc.somar(2, 2);  
  
    }  
  
}
```

```
        assertEquals(4, resultado);  
    }  
}
```

9) Executar testes

Você pode rodar de 3 formas:

✓ Pelo botão verde

- Clique no  ao lado do método ou classe

✓ Clique direito

- Run 'CalculadoraTest'

✓ Rodar todos

- Clique na pasta `test`

10) Ver resultado

O IntelliJ mostra:

-  Verde → passou
-  Vermelho → falhou